

Collections Framework

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor (CSE), PIT
Parul University

1.	Collection Interface and Implementations.....	1
2.	List Interface and Implementations	2
3.	Summary of Key Concepts.....	4
4.	References.....	5

10.3.1 Collection Interface and Implementations

◆ 1. The Collection Interface

The Collection interface is the **root of the Collection hierarchy** in Java (except for Map). It is part of the **java.util** package and defines the basic operations that all collection types (like List, Set, Queue) should implement.

✓ Key Characteristics:

- Defines **generic container behavior** (add, remove, iterate).
- Extended by other interfaces like List, Set, Queue.

◆ 2. Common Methods in Collection Interface

Method	Description
add(E e)	Adds an element to the collection
remove(Object o)	Removes a single instance
size()	Returns the number of elements
clear()	Removes all elements
contains(Object o)	Checks if element exists
isEmpty()	Checks if collection is empty
iterator()	Returns an iterator for traversal

These methods are available in most concrete classes like ArrayList, HashSet, etc.

◆ 3. Common Implementing Classes

✓ ArrayList

- **Resizable array implementation** of List.
- Fast random access, slower for insert/delete in middle.

```
List<String> list = new ArrayList<>();
list.add("Java");
```

✓ LinkedList

- Implements both List and Deque.
- Good for insert/delete operations.

```
List<Integer> numbers = new LinkedList<>();
numbers.add(10);
```

✓ **HashSet**

- Implements Set.
- Does **not allow duplicates**, unordered.

```
Set<String> set = new HashSet<>();  
set.add("Apple");  
set.add("Banana");
```

✓ **TreeSet**

- Implements NavigableSet.
- Elements stored in **sorted order** (uses Red-Black tree).

```
Set<Integer> treeSet = new TreeSet<>();  
treeSet.add(20);  
treeSet.add(10);
```

◆ **Example Code**

```
import java.util.*;  
  
public class CollectionDemo {  
    public static void main(String[] args) {  
        Collection<String> fruits = new ArrayList<>();  
        fruits.add("Apple");  
        fruits.add("Banana");  
        System.out.println(fruits.contains("Banana")); // true  
        fruits.remove("Apple");  
        System.out.println(fruits); // [Banana]  
    }  
}
```

10.4.1 List Interface and Implementations

◆ **1. Features of List Interface**

The List interface is a part of the **java.util package** and extends the Collection interface.

✓ **Key Characteristics:**

- **Ordered collection:** Elements are stored in insertion order.
- **Index-based:** Allows access to elements using indices (like arrays).
- **Allows duplicates:** Elements can repeat.
- **Supports null elements.**

✓ Common Methods:

Method	Description
add(E e)	Adds element at the end
add(int index, E e)	Inserts at specific index
get(int index)	Retrieves element at index
set(int index, E e)	Updates element
remove(int index)	Removes element at index
size()	Number of elements

◆ 2. Implementations of List Interface

✓ a. ArrayList

- Resizable-array implementation.
- Fast random access (via get()).
- Slower insertions/deletions in the middle.
- Not synchronized.

```
List<String> names = new ArrayList<>();
names.add("Alice");
System.out.println(names.get(0)); // Output: Alice
```

✓ b. LinkedList

- Doubly linked list.
- Better for frequent insertions/deletions.
- Slower access time compared to ArrayList.
- Implements Deque interface as well.

```
List<Integer> nums = new LinkedList<>();
nums.add(5);
nums.add(10);
System.out.println(nums.remove(0)); // Removes 5
```

✓ c. Vector

- Synchronized version of ArrayList.
- Thread-safe but slower.
- Legacy class, rarely used in new code.

```
List<String> list = new Vector<>();
list.add("Java");
```

◆ 3. Accessing, Updating & Iterating Lists

✓ Accessing Elements

```
System.out.println(list.get(2)); // Retrieves element at index 2
```

✓ Updating Elements

```
list.set(1, "UpdatedValue"); // Updates value at index 1
```

✓ Iterating a List

Using For Loop:

```
for(int i = 0; i < list.size(); i++) {  
    System.out.println(list.get(i));  
}
```

Using Enhanced For Loop:

```
for(String s : list) {  
    System.out.println(s);  
}
```

Using Iterator:

```
Iterator<String> itr = list.iterator();  
while(itr.hasNext()) {  
    System.out.println(itr.next());  
}
```

➤ Summary of Key Concepts:

- Collection interface defines the **core methods** for collections.
- Common implementations include ArrayList, LinkedList, HashSet, etc.
- These classes differ in performance and ordering behavior.
- Methods like add(), remove(), contains(), and iterator() are shared across implementations.
- List maintains **insertion order** and allows **duplicates**.
- Three common implementations: ArrayList, LinkedList, Vector.
- Offers **indexed access** and various iteration options.
- Choose ArrayList for **fast access**, LinkedList for **fast insertion/deletion**, Vector for **thread safety**

➤ References:

- Oracle Docs – Java Collection Interface
 <https://docs.oracle.com/javase/8/docs/api/java/util/Collection.html>
- GeeksforGeeks – Collection Interface in Java
 <https://www.geeksforgeeks.org/collection-interface-java/>
- Baeldung – Java Collections Overview
 <https://www.baeldung.com/java-collections>
- Oracle Java Docs – List Interface
 <https://docs.oracle.com/javase/8/docs/api/java/util/List.html>
- GeeksforGeeks – List Interface in Java
 <https://www.geeksforgeeks.org/list-interface-java-examples/>
- Baeldung – Java List Interface
 <https://www.baeldung.com/java-list>

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat | **GRADE**



    
<https://paruluniversity.ac.in/>